

Project Overview: Java Project 1

- **Project Name:** "OrderFlow" - High-Throughput Matching Gateway.
- **Domain:** FinTech & Capital Markets.
- **Goal:** Build a matching engine that processes millions of buy and sell orders with microsecond latency using a lock-free architecture.

Team Roles for 5 Members

-  **Member 1 (Core Backend):** Focuses on the LMAX Disruptor and lock-free Ring Buffer.
-  **Member 2 (Logic & Testing):** Focuses on Order Book logic and latency auditing.
-  **Member 3 (Integration & API):** Focuses on Spring WebFlux and the Message Broker.
-  **Member 4 (Standard UI):** Focuses on React UI layouts and DOM rendering.
-  **Member 5 (High-Performance UI):** Focuses on WebSockets and HTML5 Canvas rendering.

Week 1: Setup & Scaffolding

- **Member 1:** Implement the LMAX Disruptor pattern in Java to manage incoming order events without using traditional locks.
- **Member 2:** Set up a basic Ring Buffer architecture to bypass standard JVM Garbage Collection overhead.
- **Member 3:** Initialize a Message Broker using Aeron or Kafka to handle ultra-low latency event broadcasting.
- **Member 4:** Build the basic React UI scaffolding and create an Order Entry form for Buy/Sell and Limit/Market orders.
- **Member 5:** Prepare the WebSocket connections in the frontend to receive streaming market data.

Week 2: Core Logic & Mid-Project Review

- **Member 1:** Build the core Order Book using primitive collections or custom Object Pools (like Eclipse Collections).
- **Member 2:** Run a Latency Audit to prove the system processes 100,000 orders per second with under 100 microseconds of latency.
- **Member 3:** Set up the Market Data Gateway using Spring WebFlux to stream trade updates.
- **Member 4:** Connect the React app to the backend to display a live-updating list of recent trades.
- **Member 5:** Perform a Streaming Check to ensure the frontend renders all trade updates without any perceived lag.

Week 3: Broadcasting & UI Performance

- **Member 1:** Write the logic to calculate the "Level 2" Order Book depth, which aggregates volume at each price level.
- **Member 2:** Configure the system to broadcast this Level 2 data every 100ms.
- **Member 3:** Stream the Order Book snapshots to clients via Server-Sent Events (SSE) or WebSockets.
- **Member 4:** Implement the Order Book UI using standard React DOM rendering.
- **Member 5:** Implement the Order Book UI using HTML5 Canvas and compare its high-frequency rendering performance against standard DOM.

Week 4: Risk Check & Final Polish

- **Member 1:** Ensure the entire backend architecture serves as a masterclass in concurrency and mechanical sympathy.
- **Member 2:** Build a simulated pre-trade Risk Management check to ensure accounts have sufficient simulated funds before matching.
- **Member 3:** Ensure the Market Data Gateway communicates instantly with other internal microservices.
- **Member 4:** Polish the React UI to create a professional financial terminal interface.

- **Member 5:** Add a graphical Depth of Market (DOM) visualizer to the UI to display buy and sell pressure.